THE HONG KONG UNIVERSITY OF SCIENCE & TECHNOLOGY
Department of Computer Science and Engineering
ARIN5103: Foundations of Machine Learning / MAIE5212: Machine Learning
Fall 2025 Assignment 2
Due time and date: 11:59pm, Nov 26 (Wed), 2025.

IMPORTANT NOTES

- **Your grade will be based on the correctness and clarity.**
- **Late submission: 25 marks will be deducted for every 24 hours after the deadline.**
- **ZERO-Tolerance on Plagiarism: All involved parties will get zero mark.**

Q1: SVM

Consider the following dataset with three training points in $\mathbb{R}^2$:

- $\mathbf{x}_1 = (1, 1)^T$, with label $y_1 = +1$,
- $\mathbf{x}_2 = (2, 2)^T$, with label $y_2 = +1$,
- $\mathbf{x}_3 = (0, 0)^T$, with label $y_3 = -1$.

You train a soft-margin SVM using a polynomial kernel of degree $d = 2$: $k(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j + 1)^2$. The regularization parameter is set to $C = 1.0$. After solving the dual quadratic program, you obtain the Lagrange multipliers:

$$\alpha_1 = 0.25, \quad \alpha_2 = 0, \quad \alpha_3 = 0.25.$$

Based on this information, answer the following questions, and show your steps clearly.

(a) Which training points are support vectors?

(b) Recall that the decision function is $f(\mathbf{x}) = \text{sign}\left(\sum_{i=1}^3 \alpha_i y_i k(\mathbf{x}_i, \mathbf{x}) + b\right)$. What is the value of b?

(c) For a point $\mathbf{x} = (x_a, x_b)^T$, write the expression $\sum_{i=1}^3 \alpha_i y_i k(\mathbf{x}_i, \mathbf{x}) + b$ as a fully-expanded polynomial:

(d) Using the function obtained in part (c), determine the label of a new data point $\mathbf{x}_{\text{new}} = (1, 0)^T$. Show your steps clearly.

Q2. K-Means Clustering

The K-Means clustering algorithm may converge to a sub-optimal solution due to its sensitivity to the initial placement of cluster centers. The standard mitigation is to run the algorithm multiple times (`n_init`) with different random initializations and select the best result. In this section, you will implement this strategy and then use your implementation to determine the optimal number of clusters for a dataset.

You are provided with the following initial code, which implements a *single run* of the K-Means algorithm.

```python
import numpy as np
from sklearn.metrics import pairwise_distances_argmin

def find_clusters(X, n_clusters, rseed=2):
    # 1. Randomly choose clusters
    rng = np.random.RandomState(rseed)
    i = rng.permutation(X.shape[0])[:n_clusters]
    centers = X[i]

    while True:
        # 2a. Assign labels based on closest center
        labels = pairwise_distances_argmin(X, centers)

        # 2b. Find new centers from means of points
        new_centers = np.array([X[labels == i].mean(0) for i in range(n_clusters)])

        # 2c. Check for convergence
        if np.all(centers == new_centers):
            break
        centers = new_centers

    return centers, labels
```

1. **Modify the Provided Code to Calculate SSE:** The provided `find_clusters` function is missing a crucial component: it does not calculate the objective of K-means (i.e., the sum squared error SSE). Modify the function so that it calculates and returns the SSE in addition to the centers and labels. The function signature should become: `def find_clusters(X, n_clusters, rseed=2): ... return centers, labels, sse`.

2. **Implement the `n_init` Strategy:** Write a **new** function called `find_best_clusters(X, n_clusters, n_init)`. This function will implement the logic of running K-Means multiple times to find the best outcome.

   - Inside this function, create a loop that runs `n_init` times.
   - In each iteration, call your modified `find_clusters` function from Step 1. Use the loop index as the `rseed` to ensure a different starting point for each run.
   - Keep track of the best result seen so far. Compare the SSE of the current run with the best SSE found in previous runs. If the current run has a lower SSE, store its centers, labels, and SSE as the new "best."
   - After the loop completes, the function should return only the best set of centers, labels, and the corresponding lowest SSE found across all `n_init` runs.

3. **Apply the Elbow Method with Your Implementation:**

   - First, generate a dataset suitable for this analysis. Use `make_blobs` from Scikit-Learn with `n_samples=300`, `centers=5`, a `cluster_std` of 1.0, and a `random_state` of 42.
   - Write a loop that iterates through a range of cluster counts, k, from 2 to 9 (inclusive).
   - In each iteration of the loop, call your robust `find_best_clusters` function with the current value of k. Use `n_init=10` to ensure you find a good solution for each k.
   - Store the returned (best) SSE for each k in a list.

4. **Plot and Analyze the Elbow Curve:**

   - Create a line plot with the number of clusters (k) on the x-axis and the corresponding best SSE on the y-axis.

- Identify the "elbow" point on the graph. Based on your plot, what is the optimal number of clusters for this dataset?
- Explain the trade-off that the elbow plot illustrates. Why does SSE always decrease as k increases, and what makes the"elbow" the point of interest?

Submission Guidelines

Please submit a pdf (A2.pdf) containing your answers to Q1 and a Python notebook (A2.ipynb) containing code, running outputs and discussions for Q2.
Please submit the assignment by uploading the compressed file to Canvas. Note that the assignment should be clearly legible, otherwise you may lose some points if the assignment is difficult to read. Plagiarism will lead to zero point on this assignment.